When Every Millisecond Costs Money.

SLOs for Revenue-Critical Systems

Chenghao Liu
AI Builder | ex-Meta AI Engineer | ex-Microsoft | ex-Amazon | GoodRx | Startup Founder

SREDay Seattle

When 100ms = Millions.

*Why **SLO** became a **revenue** problem.*

SREDay @Seattle

SREDay

100 milliseconds.

Enough to move a multi-billion-dollar business.

+100ms latency → **−1%** revenue

Documented across Amazon, Meta, Microsoft. The pattern is not an anomaly — it is the baseline physics of the web.

CONTEXT

From SLAs to SLOs to Revenue.

Twenty years of reliability, measured three different ways.

2005

SLAs

Contracts.

Binary. Legal. Measured in quarterly credits, not engineer behavior.

2015

SLOs

Error budgets.

Percentiles. Internal. Great for platform teams — blind to business weight.

2025

REV-AWARE

Dollars at risk.

Cohort-weighted. Journey-aware. Denominated in revenue per minute.

SLO is not a technical metric anymore.

It ties directly to growth, risk, and data.

OLD VIEW

A technical measurement

- uptime
- latency
- error rate



connects them

NEW VIEW

A business signal

- **growth**
- **data**
- **risk**

THE GAP

Most systems are measured wrong.

ENGINEERING DASHBOARD

All green.

Uptime	99.97%
p99 latency	187ms
Error rate	0.04%
CPU	42%

BUSINESS DASHBOARD

Quietly bleeding.

Revenue	−$420K this qtr
Conversion	−1.8% WoW
Top-decile spend	−6%
Alerts fired	0

We measure what's easy to graph. The business measures what pays the bills.

Usually, people only realize it when it is down.

Meta, 2022 — a SEV0, company-wide outage.

SEV0 Company-wide host outage. Every service degraded.

8 HOURS
to full recovery

ALL SCOPE
impacted company-wide

1 LEAK
recovery doc shared publicly during the fix

No dashboard warned them before — and none of it was in the **SLO**.

THE RIGHT SLO

What the right SLO unlocks.

Three forces a well-defined SLO balances — you cannot maximize all three at once.

01

Reduce losses

Prevent transaction loss. Build disaster recovery structure and isolation into the system.

02

Protect core experience

Guard the flows the customer feels — high-quality data, fast response.

03

Ship faster with confidence

A clear error budget is permission to ship. Iteration fuels growth.

THE BALANCE

The work is finding the **balance point** where all three deliver maximum combined value.

THE RIGHT SLO

What the right SLO unlocks.

Three forces a well-defined SLO balances — you cannot maximize all three at once.

01

Reduce losses

Prevent transaction loss. Build disaster recovery structure and isolation into the system.

02

Protect core experience

Guard the flows the customer feels — high-quality data, fast response.

03

Ship faster with confidence

A clear error budget is permission to ship. Iteration fuels growth.

THE BALANCE

The work is finding the **balance point** where all three deliver maximum combined value.

ANATOMY

A revenue-critical pipeline.

Seven hops. Seven teams. One dollar at stake.



FAILURE STORY #2 ·

The day latency spiked — and revenue dropped with it.

T+0h

Dashboards green.

Recs p95 420ms · inside 500ms SLO · no pages.

T+1h

Conversion −18%.

Business slack: "is something wrong with site?"

T+2h

Top-decile p95 = 1.4s.

Aggregate hid a degraded shard serving high-LTV users.

T+3h

$1.1M of lost orders.

Rollback. No alert had fired the entire time.

Revenue concentration ≠ traffic concentration.

POSTMORTEM

Why traditional SLOs failed us.

01

Aggregates hide high-value segments

p95 is a story about the median user. Your revenue isn't.

02

Percentiles don't weight by dollars.

A 500ms miss on a $5 user ≠ a 500ms miss on a $500 user.

03

Latency SLOs are not conversion SLOs.

Fast ≠ working. The service can be healthy and the funnel still collapsing.

Every metric is a model. Every model leaks the thing you didn't model.

THE PIVOT

Redefine what you measure.

From availability → business impact.

BEFORE

99.95% uptime
p99 < 200ms
<0.1% 5xx error rate
Ship within budget



AFTER

- < $5K revenue-at-risk / hour
- < 0.2% top-decile conversion drop
- < 60s to revenue-weighted rollback
- Finance and SRE use the same dashboard

Same discipline. Different denominator.

WHAT TO TRACK

Work backwards on SLO tracking.

Each layer answers a different question — you need all four for observability & ops

METRICS	*What is the system doing?*	Counters, rates, latencies. The always-on pulse — ideal for dashboards, alerts, and the SLO itself.
LOGGING	*What happened, and why?*	Discrete events with context. Replay an incident, explain anomalies, and audit a single request end-to-end.
TRACING	*Where did the time go?*	A request's path across services. Pinpoint the slow hop in a distributed system and find the real bottleneck.
PROFILING	*Why is this host expensive?*	CPU, memory, flame graphs. Attribute cost and tune hot paths inside a single process.

OBSERVABILITY

You can't fix what you can't see.

The minimum signal set for revenue-critical systems.

Revenue / minute	Real-time, baselined against 7-day & same-day-last-week.
Funnel dropout by segment	Per-stage conversion, segmented by revenue. Anomaly detect per segment, not in aggregate.
Model output distribution	KL divergence vs. training baseline. Watch the shape, not just the mean.
Revenue-weighted rollout	Route 1% of traffic, but 10% of revenue, to test. Rollout what matters.

THE RIGHT SLO

What the right SLO unlocks.

01

Reduce losses

Prevent transaction loss. Build disaster recovery structure and isolation into the system.

02

Protect core experience

Guard the flows the customer feels — high-quality data, fast response.

03

Ship faster with confidence

A clear error budget is permission to ship. Iteration fuels growth.

THE BALANCE

The work is finding the **balance point** where all three deliver maximum combined value.

FAILURE STORY #3 · Data Drift Loss

The model was "healthy."
The business was not.

ML HEALTH

AUC	0.842 (stable)
Input drift	p < 0.03
Infer p99	41ms
Feature freshness	OK

BUSINESS HEALTH

Add-to-cart	−7% / 2wk
Avg order value	−12%
Margin per session	−9%
Alerts fired	0

The model was optimizing the wrong thing — perfectly.

PROBLEM RCA

Before fixing — ask which layer.

Structured hypotheses > blind debugging. We bucketed into three.

01

INFRA

Lingering degradation?

Tail latency after recovery · partial service failure · regional imbalance

CHECKED — clean

02

DATA / PIPELINE

Missing or stale inputs?

Feature freshness · completeness · cross-region alignment · schema drift

DATA MISMATCH

03

MODEL / CONFIG

Version or weight drift?

Mismatched deploy · corrupted weights · feature-mapping shift

CHECKED — clean

Narrow first, then dig. Every minute of triage structure saves an hour of log-diving.

FAILURE STORY #3 GOODRX

When the model was fine — and quietly, very wrong.

A pharmacy pricing model, 20% accuracy drop nobody paged.

WHAT THE MODEL DOES

Predicts location-based drug discounts

Ranks best pharmacy option per user

Runs on real-time pricing + inventory

WHAT IT DIRECTLY MOVES

User savings — per-script

Conversion — click-to-fill rate

Revenue — direct commercial line

WHAT HAPPENED

SEV0 recovered.
The model didn't.

INFRA VIEW

"System is back to normal."

Latency	stable
APIs	2xx across the board
Load	normal
Alerts fired	0

REALITY

Accuracy −20 to −30%.

Wrong discounts predicted	
Suboptimal pharmacy ranking	
User savings	degraded
Revenue	noticeably down

Every dashboard was green. Every dollar was quietly red.

ROOT CAUSE

The pipeline didn't fully resync.

Recovery finished the services. It didn't finish the data.

DURING RECOVERY

A feature pipeline dependency failed to fully resync.

Real-time pricing inputs became stale · partially missing · region-misaligned.

The model kept running — still "accurate" on its old assumptions.

But it was now operating on invalid real-world inputs.

The model was still accurate — on data that no longer existed.

FIX & TAKE-HOMES

From revenue to data-aware SLOs.

IMMEDIATE

Stop the bleed.

- Restore feature-pipeline sync
- Fallback to last-known-good feature snapshot

SHORT-TERM

Make the silent loud.

- Feature-completeness checks pre-inference
- Block predictions when critical inputs missing

LONG-TERM

Data-aware SLOs.

- Freshness + drift as first-class SLIs
- Integrated into production alerting

ML SLOs aren't just latency and error rate. They're what the model is actually seeing.

What the right SLO unlocks.

Three forces a well-defined SLO balances — you cannot maximize all three at once.

01

Reduce losses

Prevent transaction loss. Build disaster recovery structure and isolation into the system.

02

Protect core experience

Guard the flows the customer feels — high-quality data, fast response.

03

Ship faster with confidence

A clear error budget is permission to ship. Iteration fuels growth.

THE BALANCE

The work is finding the **balance point** where all three deliver maximum combined value.

SHIP FASTER WITH CONFIDENCE

Error budgets

When there is still budget, system stable, ship fast.

SIGNAL

SLO is green

Budget remaining. No freeze, no war room. Eng owns the call.

BEHAVIOR

Releases go up

Smaller, more frequent deploys. Faster feedback, smaller blast radius.

OUTCOME

Revenue grows

More experiments reach users. Winning features convert sooner.

2× release cadence = 2× learning rate = revenue you didn’t have last quarter.

The SLO doesn’t just prevent outages — it unlocks speed.

SYSTEM DESIGN

Resilient recovery loops.

Infra + ML + Product as one feedback system.

DETECT

Business signals first.

Rev/min, revenue segments, model output drift — not just latency.

→

DECIDE

Dollar-denominated rules.

e.g. if rev/min < baseline − 3σ for 60s → auto-fallback.

→

ACT

Graceful degrade paths.

Fallback model, static pricing, cached recs — tested monthly.

↺ *Post-incident → replay as test case*

THE RIGHT SLO

What the right SLO unlocks.

Three forces a well-defined SLO balances — you cannot maximize all three at once.

01

Reduce losses

Prevent transaction loss. Build disaster recovery structure and isolation into the system.

02

Protect core experience

Guard the flows the customer feels — high-quality data, fast response.

03

Ship faster with confidence

A clear error budget is permission to ship. Iteration fuels growth.

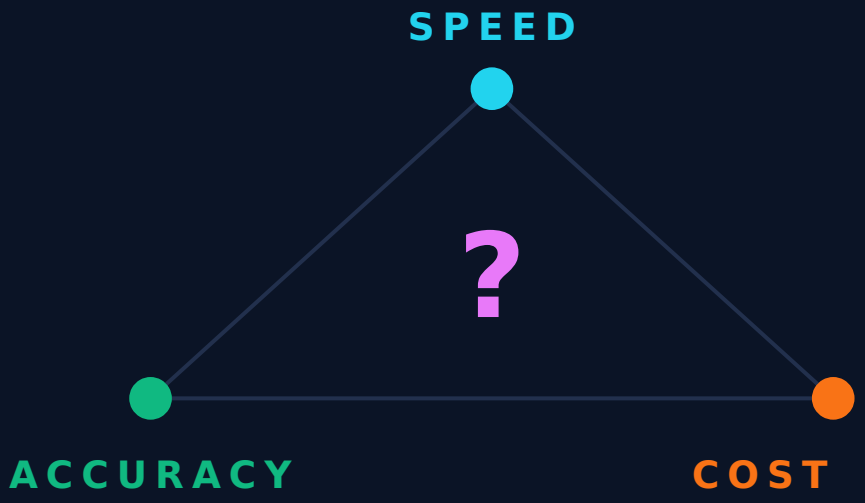
THE BALANCE

The work is finding the **balance point** where all three deliver maximum combined value.

THE QUESTION

How do you find the balance point?

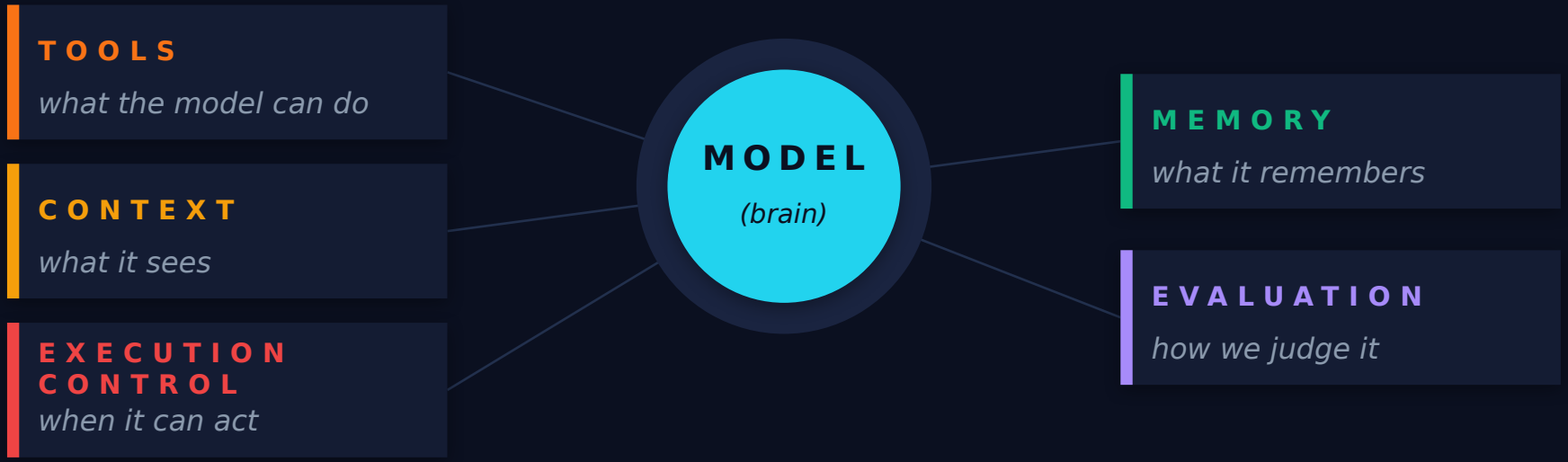
Three forces. One dial. An impossible triangle — unless something changes.



HARNESS ENGINEERING

The system around the model.

The model is not the system — the harness is.



NEXT / A NEW WAY TO RUN SLOs

Beyond error budgets. Harness Engineering.

Error budgets tell you when to stop. Harnesses keep you running.

NOW POSSIBLE — BECAUSE OF AI

- **LLMs read logs.** Triage in seconds, not shifts.
- **Agents remediate.** Auto-rollback, auto-mitigate.
- **Models predict.** Forecast burn, don’t react to it.

PRECISE DEFINITION · 02

Harness = runtime control system.

The decision-making and execution layer wrapped around an AI.



↺ *feedback · world → harness (never to the model directly)*

WHAT IT GOVERNS

verification · routing · rollback · cost ceilings · policy

WHY IT MATTERS

the model proposes; the harness is what's liable in production.

HARNESS FOR SLO

The SLO harness.

The control plane for reliability



TRADITIONAL SLO

Monitor → alert → human decides → action

Dashboards. Pages. Runbooks.

HARNESS-DRIVEN SLO

Sense → reason → decide → act → learn

A control loop. Continuous. Accountable.

THE FIVE JOBS

What jobs harness actually does.

A closed-loop control system — five interlocking jobs.

1 Signal Collection & Normalization

Ingest telemetry, biz, data signals. One schema. One clock. Honest baselines.

2 Decision Structure

Rules, models, policies combined — not a prompt, a program. Explainable outputs.

3 Verification — Before & After

Gate the action pre-execution. Measure its effect post-execution. Never skip either side.

4 Memory & State

Remember last incident, last rollback, last anomaly. Context beats cleverness.

5 Learning Loops

Every action becomes evidence. The harness improves — the model doesn't have to.

`Sense → decide → verify → act → remember → learn ↺`

HARNESS vs ERROR BUDGET

Static budgets → adaptive control.

Error budgets define the limits. The harness enforces and optimizes them.

TRADITIONAL ERROR BUDGETING	AI HARNESS FOR SLO
Static thresholds	Dynamic, context-aware
Uptime / latency only	Multi-layer: infra · data · business
Alert → human decides	Detect → decide → act
Reactive	Proactive + predictive
No memory	Learns from incidents
Binary compliance	Continuous optimization
Focuses on system health	Focuses on business outcomes

Error budgets define the boundary. The harness is what keeps you inside it.

TAKEAWAYS

Six things to take home.

What you measure → which signals count → how you act.

MEASURE IN DOLLARS
what you measure

Dollarize every SLO.
Budget in $/hr, not percentiles. Finance and SRE finally share a dashboard.

Segment every metric.
Aggregate metrics hide the users who pay the bills. Pivot by revenue cohort.

WIDEN THE SIGNAL
which signals count

Find the balance point
Reduce loss, improve data & experience, iterate updates & innovation

Service health ≠ outcome health.
Green dashboards can hide a broken funnel. Co-measure both, every time.

AUTOMATE THE LOOP
how you act

Recovery is a pipeline.
Detect → decide → verify → act → learn. Runbooks don't scale; control loops do.

Build the control panel
Error Budge / Harness. Models propose. Reliability compounds in the harness.

Tie SLOs to dollars.

TAKEAWAYS FRAMEWORK

Designing revenue-aware SLOs.

01

Weight users by revenue.

Cohort-based percentiles. Your top decile gets its own target.

02

Slice by journey stage.

Checkout ≠ browse. Different SLOs, different budgets, different alerts.

03

Budget in dollars per hour.

Stop saying 99.9%. Start saying "$X exposed per 30 min of outage."

04

Rollback on revenue, not latency.

Trigger auto-rollback from business signal, not CPU.

CLOSING THOUGHT

Engineers build the runway.
Revenue is the landing strip.

If you don’t tie SLOs to revenue, you’re flying blind.

LET'S TALK

Questions.

https://www.linkedin.com/in/liuchenghao/

Love to hear what you experienced as well. Three things I'd love to hear: the worst metric your team still trusts; your favorite fallback path; where revenue-aware breaks down



LET'S TALK

Thank you

https://www.linkedin.com/in/liuchenghao/

